# UNIDAD 2
# ESTRUCTURAS DE CONTROL

Por Félix Ferreiras
y Zobiesky Ovalle

Decisiones, ciclos

# Objetivos

En esta unidad usted aprenderá a:

- Crear programas computacionales utilizando las diferentes estructuras de control que maneja C++
- Usar los operadores relacionales y condicionales
- Crear instrucciones utilizando las sentencias IF e IF / ELSE
- Utilizar la sentencia SWITCH
- Crear bucles mediante la sentencia WHILE
- Desarrollar bucles FOR
- Crear bucles mediante la sentencia DO / WHILE
- Implementar las sentencias complementarias (break, continue).

# Contenido

- Las estructuras de control.
- Sentencia de decisión (if) y operador condicional (?:).
- Sentencia de decisión (switch).
- Sentencia iterativa (while).
- Sentencia iterativa (do-while).
- Sentencia iterativa (for).
- Complementarias (break, continue).

# Contenido

- Las estructuras de control.
- Sentencia de decisión (if) y operador condicional (?:).
- Sentencia de decisión (switch).
- Sentencias iterativas (while, do-while, for).
- Complementarias (break, continue).

# Las estructuras de control

Decisiones, ciclos

# Estructuras de control

- Las estructuras de control permiten al programador cambiar la secuencia de ejecución de las sentencias
- C++ al igual que cualquier otro lenguaje de programación, dispone de un conjunto de sentencias para tomar decisiones, realizar ciclos y hacer selecciones dentro de un programa

# Estructuras de control

| Clasificación | Sentencias | Comentarios |
|---|---|---|
| Decisión | `if` | Toma de decisiones, puede escrita de diversas formas |
| | `?:` | Operador de toma de decisiones |
| Ciclos o bucles | `while` | Ciclo que permite ejecutar una o mas sentencias, evalúa al inicio |
| | `do-while` | Ciclo que permite ejecutar una o mas sentencias, evalúa al final |
| | `for` | Estructura que permite ejecutar una cantidad predefinida de ciclos |
| Selectivas | `switch` | Sentencia selectiva, toma de decisiones, no permite rangos |
| Bifurcación | `break`<br>`continue` | Se pueden combinar con sentencias de ciclo y el break con la sentencia switch |

# Contenido

- Las estructuras de control.
- Sentencia de decisión (if) y operador condicional (?:).
- Sentencia de decisión (switch).
- Sentencias iterativas (while, do-while, for).
- Complementarias (break, continue).

# Sentencia if y operador condicional

Sentencia if y operador condicional ?:

# Condición o expresión lógica

- Es toda expresión que al ser evaluada retorna un valor de verdad falso o verdadero
- Es muy importante conocer como estructurar las expresiones lógicas pues son muy utilizadas en combinación con otras sentencias que requieren en su sintaxis el uso de dichas condiciones
- Otro razón de igual es importancia es que estas expresiones o condiciones forman parte de la lógica de los algoritmos que diseñamos, pudiendo ser estas la razón de un error de lógica si no están escritas de acuerdo a lo que persigue la solución del problema

# Condición o expresión lógica

- Una expresión o condición lógica puede estar compuesta por operadores y operándoos de distintos tipos pero al ser evaluada debe retornar un valor lógico (true / false)
- Se debe manejar a la perfección principalmente los operadores lógicos (&&, ||, !), conjunción, disyunción y negación respectivamente

# Condición o expresión lógica

## Ejemplos

1.
```
int a = 0;
bool encontrado = false;
bool res;
res = (a > 0 || encontrado);
```
2.
```
double precio = 23.45;
bool aplicaITBIS = true;
bool r;
r = (precio > 50.0 && !aplicaITBIS);
```
3.
```
bool esPar;
int num = 45;
esPar = (num % 2 == 0);
```

## Contra - Ejemplos

1.
```
int a = 0;
bool seguir = false;
bool res;
res = (a >= seguir);
```
2.
```
double altura = 2.8;
bool estado = true;
bool apl;
apl = (altura + estado < 8.0);
```
3.
```
bool existe = false;
long factorial = 120;
bool resultado;
resultado = (factorial && existe);
```

# Sentencia if

- La sentencia if nos permite bifurcar el flujo o ejecución del programa según el resultado de la condición lógica evaluada
- Ramifica la ejecución secuencial en función del resultado de evaluar una condición booleana
- Dependiendo del valor resultante de evaluar la expresión lógica (true / false) el flujo del programa se desviará hacia uno u otro punto de las sentencias
- La sentencia if puede ser estructurada de distintas formas:
  - if simple
  - if doble
  - if múltiple

# Sentencia if (simple)

- if (simple)

```
if (condicion_logica)
{
    sentencia 1;
    sentencia 2;
    sentencia N;
}
```

**Concepto**

En esta sintaxis todas las sentencias dentro de las llaves serán ejecutadas si la condición lógica retorna un valor de verdad verdadero, de lo contrario este bloque de código será obviado y el programa continuará en la próxima línea luego de la llave que cierra.

Si la condición lógica es verdadera, el flujo continúa también después de la llave que cierra luego de ejecutar todas las sentencias del bloque.

- if (simple)

```
if (condicion_logica)
    sentencia;
```

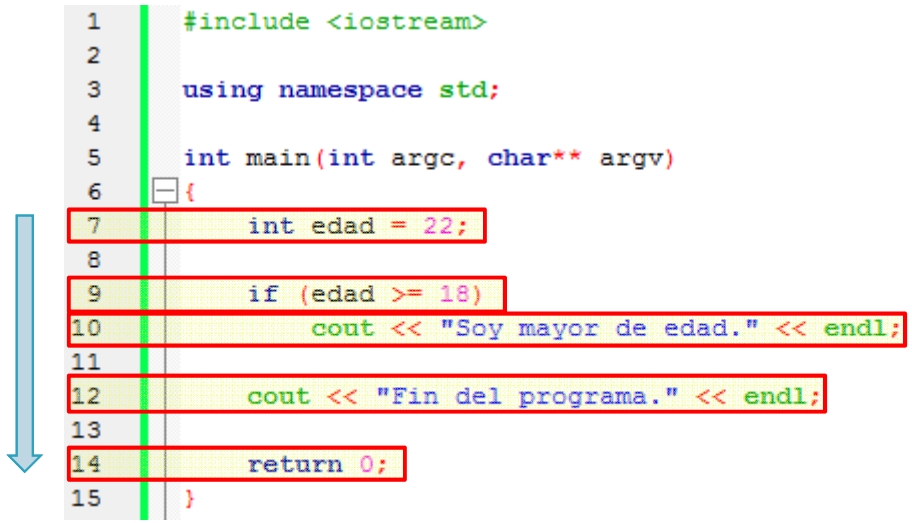
**Concepto**

En esta sintaxis no hay un bloque de código que agrupe una o más sentencias, esto es por que si la cantidad de sentencias que serán ejecutadas cuando la condición lógica sea verdadera es igual a uno, las llaves son opcionales, no obstante para mayor claridad puede optar por colocar las llaves y así queda expresado de una forma más explícita.

Si la condición lógica es verdadera, la sentencia siguiente al ***if*** es ejecutada y el flujo continúa con la siguiente sentencia. Si la condición lógica es falsa, la sentencia siguiente al ***if*** es obviada y el flujo del programa continúa con la sentencia que se encuentra luego del punto y coma (;) que termina la sentencia más próxima al ***if***.

# Sentencia if (simple)

- Veamos unos ejemplos que expliquen mejor flujo de ejecución del programa (caso cuando el resultado es *true*)



```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int edad = 22;
8
9       if (edad >= 18)
10          cout << "Soy mayor de edad." << endl;
11
12      cout << "Fin del programa." << endl;
13
14      return 0;
15  }
```

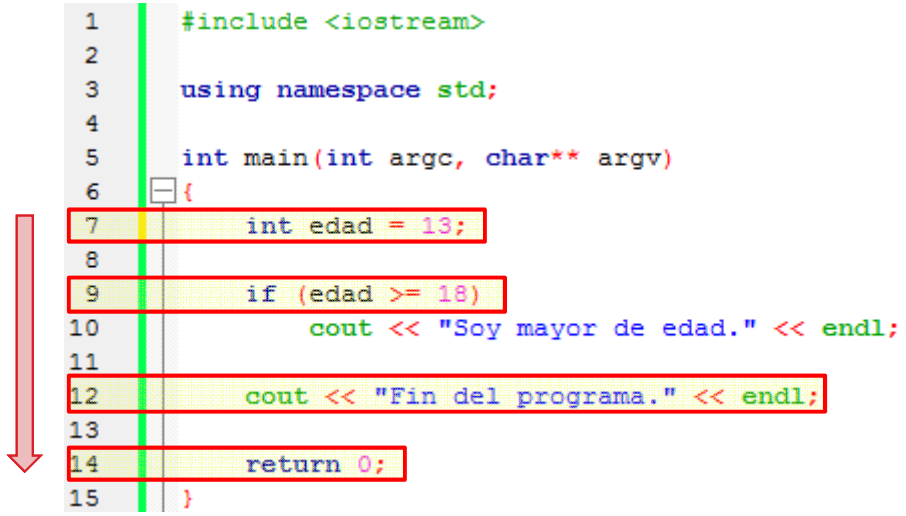
**Caso cuando la condición lógica al ser evaluada retorna un valor de verdad verdadero**

En este programa, si la variable edad es mayor o igual que 18 (lo cual es así), las líneas y su secuencia dentro de la función ***main*** que serían ejecutadas son:

**7, 9, 10, 12, 14**

# Sentencia if (simple)

- Veamos unos ejemplos que expliquen mejor flujo de ejecución del programa (caso cuando el resultado es *false*)



```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int edad = 13;
8
9       if (edad >= 18)
10          cout << "Soy mayor de edad." << endl;
11
12      cout << "Fin del programa." << endl;
13
14      return 0;
15  }
```

**Caso cuando la condición lógica al ser evaluada retorna un valor de verdad falso**

En este programa, si la variable edad **no** es mayor o igual que 18 (lo cual es así), las líneas y su secuencia dentro de la función ***main*** que serían ejecutadas son (a diferencia del ejemplo anterior, la línea 10 es obviada):

**7, 9, 12, 14**

# Sentencia if (simple)

- Veamos unos ejemplos que expliquen mejor flujo de ejecución del programa (caso cuando el resultado es *true*)

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       double ventas = 825350.00;
8       double incentivo = 0;
9       const int TASA_INCENTIVO = 4;
10
11      if (ventas >= 500000.0)
12      {
13          incentivo = ventas * (TASA_INCENTIVO / 100.0);
14          cout << "Usted ha ganado un incentivo especial."
15               << "\nEl monto de su incentivo es: "
16               << incentivo << endl;
17      }
18
19      cout << "Las ventas fueron: " << ventas << endl;
20
21      return 0;
22  }
```

**Caso cuando la condición lógica al ser evaluada retorna un valor de verdad verdadero (usando if con llaves)**

En este programa, si la variable sueldo es mayor o igual que 500,000 (lo cual es así), las líneas y su secuencia dentro de la función ***main*** que serían ejecutadas son:

**7, 8, 9, 11, 13, 14, 15, 16, 19, 21**

# Sentencia if (simple)

- Veamos unos ejemplos que expliquen mejor flujo de ejecución del programa (caso cuando el resultado es *true*)

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       double ventas = 325350.00;
8       double incentivo = 0;
9       const int TASA_INCENTIVO = 4;
10
11      if (ventas >= 500000.0)
12      {
13          incentivo = ventas * (TASA_INCENTIVO / 100.0);
14          cout << "Usted ha ganado un incentivo especial."
15               << "\nEl monto de su incentivo es: "
16               << incentivo << endl;
17      }
18
19      cout << "Las ventas fueron: " << ventas << endl;
20
21      return 0;
22  }
```

**Caso cuando la condición lógica al ser evaluada retorna un valor de verdad verdadero (usando if con llaves)**

En este programa, si la variable sueldo **no** es mayor o igual que 500,000 (lo cual es así), las líneas y su secuencia dentro de la función ***main*** que serían ejecutadas son:

**7, 8, 9, 11, 19, 21**

# Sentencia if (simple)

- Tome en cuenta que la sentencia ***if*** simple solamente tiene un solo *scope* o *alcance* (espectro de acción)
- Es muy importante que se comprenda su modo de acción pues no contiene un escenario para cuando el resultado de la condición lógica es falso, sino que simplemente sale a la próxima sentencia luego de terminar el ***if***

# Sentencia if (doble)

- if (doble)

```
if (condicion_logica)
{
    sentencia 1;
    sentencia N;
}
else
{
    sentencia 1;
    sentencia N;

}
```

**Concepto**

En esta sintaxis todas las sentencias dentro del primer bloque de código serán ejecutadas si la condición lógica retorna un valor de verdad verdadero, de lo contrario este bloque de código será obviado y el programa ejecutará el segundo bloque de código, el siguiente a la cláusula ***else***.

En ambos casos, ya sea verdadero o falso y luego de ejecutar el bloque de código correspondiente, el flujo del programa continuará en la siguiente línea después de la última llave.

- if (doble)

```
if (condicion_logica)
    sentencia;
else
    sentencia;
```

**Concepto**

En esta sintaxis no hay bloques de código que agrupe una o más sentencias, esto es por que si la cantidad de sentencias que serán ejecutadas cuando la condición lógica sea verdadera o falsa es igual a uno, las llaves son opcionales, no obstante para mayor claridad puede optar por colocar las llaves y así queda expresado de una forma más explícita.

Si la condición lógica es verdadera, la sentencia siguiente al ***if*** es ejecutada y el flujo continúa con la línea de código luego de la sentencia del ***else***. Si la condición lógica es falsa, la sentencia siguiente al ***if*** es obviada y es ejecutada la sentencia del ***else***. En ambos casos, el flujo continúa luego de ejecutar la sentencia correspondiente con la línea siguiente al final del ***if*** completo

# Sentencia if (doble)

- if (doble)

```
if (condicion_logica)
{
    sentencia 1;
    sentencia N;
}
else
    sentencia;
```

- if (doble)

```
if (condicion_logica)
    sentencia;
else
{
    sentencia 1;
    sentencia N;
}
```

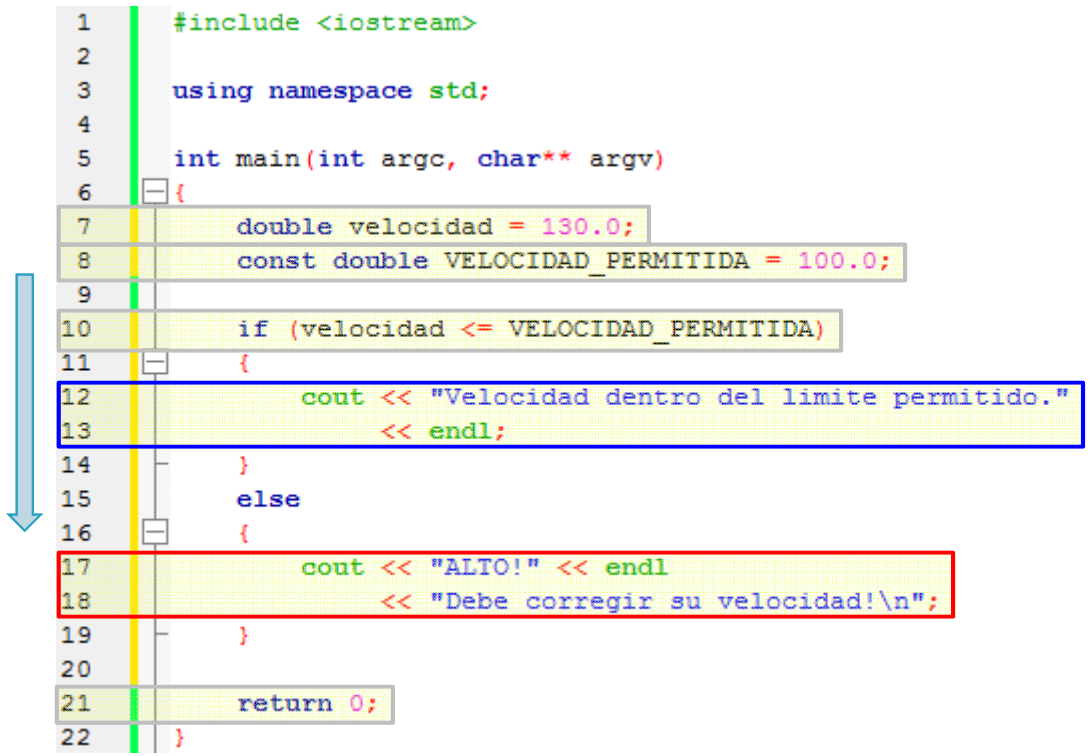
**Interesante**

Otras combinaciones que pueden utilizarse consisten en: aplicar llaves siempre y cuando necesite colocar más de una sentencia en un bloque de código y no usar llaves si en la parte del ***if*** o del ***else*** necesita solo una.

De igual forma, si esto presenta confusión, resuélvalo aplicando llaves para ambos bloques, aún teniendo solamente una sentencia ambos.

# Sentencia if (doble)

- Veamos un ejemplos que expliquen mejor flujo de ejecución del programa usando esta sintaxis



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      double velocidad = 130.0;
8      const double VELOCIDAD_PERMITIDA = 100.0;
9
10     if (velocidad <= VELOCIDAD_PERMITIDA)
11     {
12         cout << "Velocidad dentro del limite permitido."
13              << endl;
14     }
15     else
16     {
17         cout << "ALTO!" << endl
18              << "Debe corregir su velocidad!\n";
19     }
20
21     return 0;
22 }
```

En este programa, si la variable *velocidad* es menor o igual que el valor contenido en la constante VELOCIDAD_PERMITIDA entonces se ejecutará el primer bloque del ***if***, de lo contrario, o sea, si es mayor que la constante, entonces el bloque del ***if*** será obviado (el primer bloque de código) y la ejecución saltará al segundo bloque de código o bloque del ***else***.

En resumen, estas serían las líneas a ejecutar dentro de la función ***main*** para ambos casos.

**True : 7, 8, 10, 12, 13, 21**
**False : 7, 8, 10, 17, 18, 21**

# Sentencia if (múltiple)

- if (múltiple)

```
if (condicion_logica_1)
{
    sentencia 1;
    sentencia N;
}
else if (condicion_logica_2)
{
    sentencia 1;
    sentencia N;

}
else
{
    sentencia 1;
    sentencia N;
}
```

- if (múltiple)

```
if (condicion_logica_1)
    sentencia;
else if (condicion_logica_2)
    sentencia;
else
    sentencia 1;
```

**Concepto**

Esta sintaxis es muy útil para evaluar rangos de valores, escribiendo la sentencia ***if*** de esta forma podemos aplicar distintas condiciones lógicas, tantas condiciones necesitemos, y colocar la sentencia o bloque de sentencias que aplique para cada caso, es decir, la sentencia o grupo de sentencias debajo solamente se ejecutará si la condición lógica que la antecede es verdadera.

Finalmente podemos colocar la cláusula ***else*** para incluir la sentencias o bloque de código que nos interese ejecutar cuando no se cumplen ninguna de las condiciones escritas anteriormente.

La ejecución del programa continúa al final del ***if*** una vez se haya cumplido con por lo menos una de las condiciones lógicas o en su defecto con la sentencia o el bloque de código correspondiente a la cláusula ***else***.

# Sentencia if (múltiple)

- Veamos un ejemplos que expliquen mejor flujo de ejecución del programa usando esta sintaxis

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      unsigned int velocidad = 3700;
8      string mensaje;
9
10     if (velocidad >= 0 && velocidad < 300)
11         mensaje = "Es un Automovil";
12     else if (velocidad >= 300 && velocidad < 500)
13         mensaje = "Es un Helicoptero";
14     else if (velocidad >= 500 && velocidad < 1200)
15         mensaje = "Es un Barco";
16     else if (velocidad >= 1200 && velocidad < 2000)
17         mensaje = "Es un Avion";
18     else
19         mensaje = "Waooo! es Superman!!";
20
21     cout << mensaje << endl;
22
23     return 0;
24 }
```

En este programa, la variable *velocidad* tiene como valor 3700, por lo que al ser evaluada en la primera condición lógica el resultado es falso, lo mismo pasa en las demás condiciones, por lo tanto no son ejecutadas ninguna de las sentencias definidas para los rangos de valores de velocidad, sino que solamente se ejecuta la del bloque de la cláusula ***else***, esto por que no se cumplió ninguna de las expresiones evaluadas.

La ejecución por línea sería como sigue:

**False : 7,8,10,12,14,16,19,20,21**

# Operador condicional (? : )

- Sintaxis 1

```
var = (condicion_logica) ? expresion1 : expresion2;
```

**Concepto sintaxis 1**

**var** : es una variable cualquiera
**condicion_logica**: es una condición que al ser evaluada retorne un valor de verdad falso o verdadero
**expresion1**: es una variable, literal, constante o expresión que al ser evaluada retorne un valor del mismo tipo que la variable **var** o de un tipo de datos al que pueda ser promovido, o sea, de menor rango que la variable. El valor retornado por expresion1 será el que se almacenara en var si la condición lógica al ser evaluada retorna un valor de verdad **verdadero**
**expresion2**: es una variable, literal, constante o expresión que al ser evaluada retorne un valor del mismo tipo que la variable **var** o de un tipo de datos al que pueda ser promovido, o sea, de menor rango que la variable. El valor retornado por expresion2 será el que se almacenara en var si la condición lógica al ser evaluada retorna un valor de verdad **falso**

# Operador condicional (? : )

- Sintaxis 2

```
(condicion_logica) ? sentencia1 : sentencia2;
```

**Concepto sintaxis 2**

En esta sintaxis no tenemos una variable a donde almacenar un valor según el resultado de la condición lógica, sin embargo podemos establecer una condición o expresión lógica y dependiendo de su resultado, ejecutar una sentencia. Este formato se parece mucho al ***if*** simple con cláusula ***else***.

**condicion_logica**: es una condición que al ser evaluada retorne un valor de verdad falso o verdadero
**sentencia1**: es una sentencia que será ejecutada si y solo si la condición lógica al ser evaluada retorna un valor de verdad **verdadero**
**sentencia2**: es una sentencia que será ejecutada si y solo si la condición lógica al ser evaluada retorna un valor de verdad **falso**

# Operador condicional (? : )

- Veamos un ejemplos que expliquen mejor flujo de ejecución del programa usando la primera sintaxis

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      int numero;
8      string mensaje;
9
10     cout << "Entre un numero: ";
11     cin >> numero;
12
13     mensaje = (numero > 0) ? "Positivo" : "Negativo o Cero";
14
15     cout << "El numero es " << mensaje << endl;
16
17     return 0;
18 }
```

En este ejemplo, la variable mensaje le será asignado un valor, "Positivo" o el literal "Negativo o Cero", dependiendo de si la condición lógica da como resultado **falso** o **verdadero**, es decir, si la variable número (leída por teclado) es positiva, entonces la variable mensaje tendrá la asignación del literal "Positivo", de lo contrario será "Negativo o Cero"

# Operador condicional (? : )

- Veamos un ejemplos que expliquen mejor flujo de ejecución del programa usando la segunda sintaxis

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      int numero;
8
9      cout << "Entre un numero: ";
10     cin >> numero;
11
12     (numero > 0) ? cout << "Positivo" : cout << "Negativo o Cero";
13
14     return 0;
15 }
```

En este ejemplo, no tenemos variable alguna involucrada en el operador condicional, en este formato podemos introducir una sentencia en la parte del operador que corresponde para un resultado **falso** y otra sentencia en la parte que corresponde para un resultado **verdadero**, ambas sintaxis son totalmente válidas en C++

# Contenido

- Las estructuras de control.
- Sentencia de decisión (if) y operador condicional (?:).
- Sentencia de decisión (switch).
- Sentencia iterativas (while, do-while, for).
- Complementarias (break, continue).

# Sentencia de decisión switch

Selección

# Sentencia switch

- switch

```
switch (expresion)
{
    case valor1:
       sentencia 1;
       sentencia N;
       [break;]
    case valor2:
       sentencia 1;
       sentencia N;
       [break;]
    case valor3:
       .
       .
       .
    default:
       sentencia 1;
       sentencia N;
       [break;]
}
```

**Concepto**

La sentencia ***switch*** tiene un funcionamiento distinto al ***if***, en este caso la sentencia evalúa lo especificado en *expresion* (lo cual debe ser una expresión entera, o sea, que al ser evaluada debe retornar un valor entero), éste valor es comparado con lo que se coloque en *valorN* ejecutando las sentencias que se encuentren debajo de la cláusula ***case*** que haga coincidencia (match). Todas las sentencias seguirán ejecutándose una después de la otra hasta encontrar una sentencia ***break*** o hasta encontrar el fin de la sentencia ***switch*** (la última llave, la que cierra).

Si no se encontró ninguna coincidencia o match entre el valor contenido en *expresion* y alguno de los valores especificados en las cláusulas ***case***, entonces el flujo del programa es bifurcado hacia la cláusula ***default*** en donde son ejecutadas todas las sentencias hasta encontrar el fin de la sentencia ***switch***. ***default*** es opcional, pero es una buena práctica utilizarlo para abarcar la posibilidad de que no haya coincidencia con los valores definidos en los ***case***.

# Sentencia switch

- Veamos un ejemplo

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      int estacion = 3;
8
9      switch (estacion)
10     {
11         case 1:
12             cout << "Primavera" << endl;
13             break;
14         case 2:
15             cout << "Verano" << endl;
16             break;
17         case 3:
18             cout << "Otono" << endl;
19             break;
20         case 4:
21             cout << "Invierno" << endl;
22             break;
23         default:
24             cout << "Parametro invalido." << endl;
25     }
26
27     return 0;
28 }
```

Como podemos observar, en este ejemplo son comparados todos las clausulas ***case*** hasta encontrar la que coincide con el valor de la expresión o variable al inicio de la sentencia ***switich.***

En este caso se ejecutaran todas las sentencias marcadas, o sea, las que pertenecen al ***case 3*** incluyendo la sentencia ***break***, esta sentencia provoca que el flujo sea interrumpido y desviado hacia fuera de la sentencia ***switch.***

Si no se colocara el ***break*** el flujo del programa continuara ejecutando las demás sentencias aunque estas pertenezcan a otro ***case.***

Veamos el próximo ejemplo.

# Sentencia switch

- Veamos un ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       unsigned int mes;
8
9       cout << "Entre el numero del mes [1-12]: ";
10      cin >> mes;
11
12      switch (mes)
13      {
14          case 1:
15          case 3:
16          case 5:
17          case 7:
18          case 8:
19          case 10:
20          case 12:
21              cout << "Tiene 31 dias." << endl;
22              break;
23          case 2:
24              cout << "Tiene 28 dias." << endl;
25              break;
26          case 4:
27          case 6:
28          case 9:
29          case 11:
30              cout << "Tiene 30 dias." << endl;
31              break;
32          default:
33              cout << "Numero de mes incorrecto." << endl;
34      }
35
36      return 0;
37  }
```

A diferencia del ejemplo anterior, la sentencia ***switch*** evalúa para la misma acción diversos casos, fíjese que si el mes es el 4to, 6to, 9no u 11vo del año, entonces se realiza la misma sentencia, no se incluye ningún comando entre los ***case*** provocando que el flujo continúe hasta encontrar una sentencia ***break***. Esto es asumiendo que para este caso, el usuario haya digitado cualquiera de los meses mencionados.

# Contenido

- Las estructuras de control.
- Sentencia de decisión (if) y operador condicional (?:).
- Sentencia de decisión (switch).
- Sentencias iterativas (while, do-while, for).
- Complementarias (break, continue).

# Sentencias iterativas

Repitiendo sentencias a través de ciclos

while, do-while, for

# Sentencias de repetición o bucles

- Repiten un bloque código de una o más sentencias (cuerpo del bucle), hasta alcanzar una condición de terminación o salida normal del bucle
- La sentencia ***break*** (utilizada anteriormente en la sentencia ***switch***) puede ser usada para salir del ciclo antes de su terminación normal
- Tipos de sentencias de repetición:
  - while
  - do-while
  - for

# Sentencia while

- while

```
while (condicion_logica)
{
    sentencia 1;
    sentencia 2;
    sentencia N;
}
```

- while

```
while (condicion_logica)
    sentencia;
```

**Concepto**

La sentencia ***while*** evalúa la condición lógica definida dentro del paréntesis, si la expresión retorna un valor de verdad verdadero, entonces el flujo del programa entra dentro del bloque de código y es ejecutado hasta el final (paréntesis cerrado) y vuelve al inicio del ciclo para evaluar nuevamente la condición lógica.

El ciclo ***while*** mantendrá la ejecución del bloque de código debajo mientras la condición lógica sea verdadera. Cuidado con los ciclos infinitos, los bucles siempre deben proveer un mecanismo o condición de terminación.

En la segunda forma de la sentencia ***while*** vemos que solamente tiene una sentencia, esto es por que los paréntesis son opcionales si el cuerpo del ciclo esta formado por una sola sentencia, si desea puede siempre utilizar llaves para evitar confusión.



El ciclo ***while*** siempre evalúa al inicio, esto significa que si la condición lógica retorna un valor de verdad falso al ser evaluada, entonces el cuerpo del ciclo no es ejecutado.

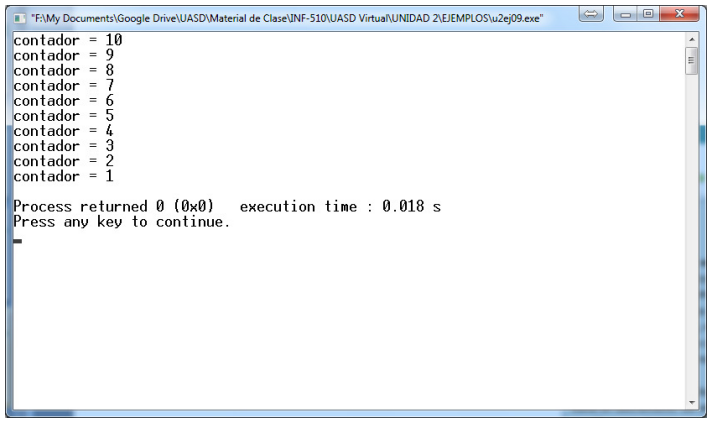


El ciclo ***while*** puede ser ejecutado cero o más veces (la cantidad de iteraciones depende de la condición o de otro factor interno dentro del cuerpo del ciclo que provoque una terminación o salida).

# Sentencia while

- Veamos un ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int contador = 10;
8
9       while (contador > 0)
10      {
11          cout << "contador = " << contador << endl;
12          contador--;
13      }
14
15      return 0;
16  }
```



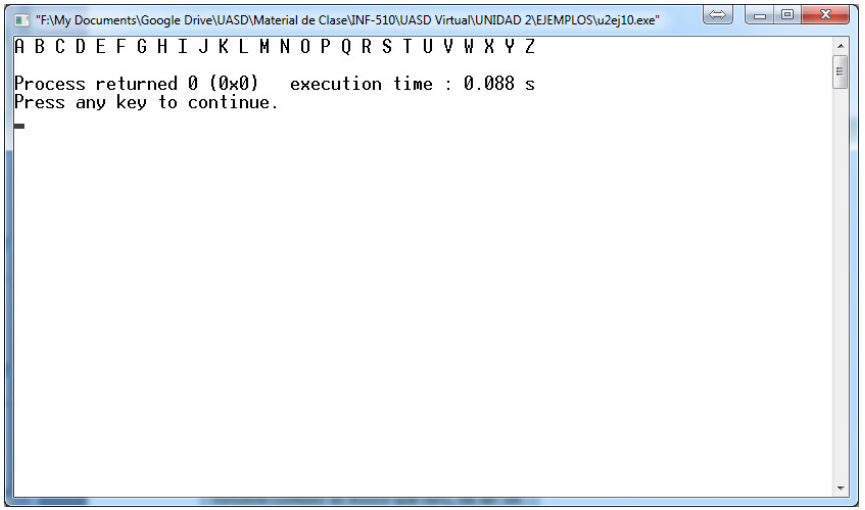

En este ejemplo, el ciclo ***while*** pregunta si la variable *contador* es mayor que cero, de ser así el flujo del programa entra al bloque de código del bucle y ejecuta todas las sentencias que allí se encuentran definidas, volviendo luego de terminar todas las líneas del cuerpo del ciclo al inicio para evaluar nuevamente la condición.

Note que en este ejemplo, la variable *contador* tiene un decremento de uno en uno, esto significa que el ciclo será ejecutado unas 10 veces.

# Sentencia while

- Veamos otro ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       char chr = 65;
8
9       while (chr < 91)
10      {
11          cout << chr << " ";
12          chr++;
13      }
14
15      cout << endl;
16
17      return 0;
18  }
```



# Sentencia do-while

- do-while

```
do
{
    sentencia 1;
    sentencia 2;
    sentencia N;
} while (condicion_logica);
```

**Concepto**

La sentencia ***do-while*** ejecuta primero el bloque de código que forma el cuerpo del ciclo y luego de esa primera ejecución evalúa la condición lógica definida dentro del paréntesis, si la expresión retorna un valor de verdad verdadero, entonces el cuerpo del ciclo o bloque de código del ***do-while*** es ejecutado nuevamente.

El ciclo ***do-while*** mantendrá la ejecución del bloque de código mientras la condición lógica sea verdadera. Cuidado con los ciclos infinitos, los bucles siempre deben proveer un mecanismo o condición de terminación.



El ciclo ***do-while*** siempre evalúa al final, esto significa que si la condición lógica retorna un valor de verdad falso al ser evaluada, entonces el cuerpo del ciclo es ejecutado una vez y luego termina.

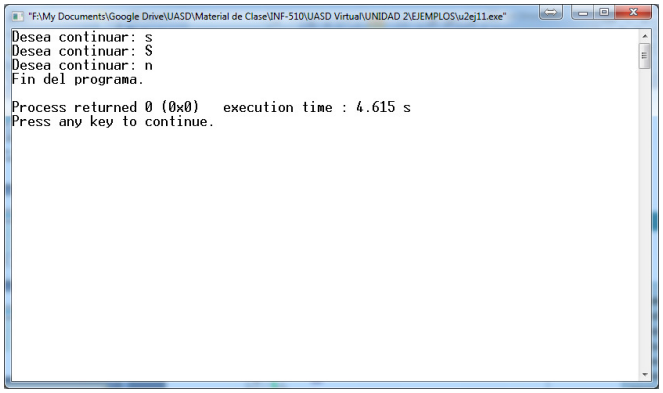


El ciclo ***do-while*** puede ser ejecutado una o más veces (la cantidad de iteraciones depende de la condición o de otro factor interno dentro del cuerpo del ciclo que provoque una terminación o salida). Esta es la diferencia entre el ciclo ***while*** y el ***do-while***.

# Sentencia do-while

- Veamos un ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       char seguir;
8
9       do
10      {
11          cout << "Desea continuar: ";
12          cin >> seguir;
13      } while (seguir == 's' || seguir == 'S');
14
15      cout << "Fin del programa." << endl;
16
17      return 0;
18  }
```




En este programa, el flujo entra directamente al bloque del ciclo ***do-while*** desplegando un mensaje y solicitando al usuario una entrada por teclado, el valor recibido es evaluado luego al final del bucle, si el resultado es verdadero entonces el ciclo se repite.

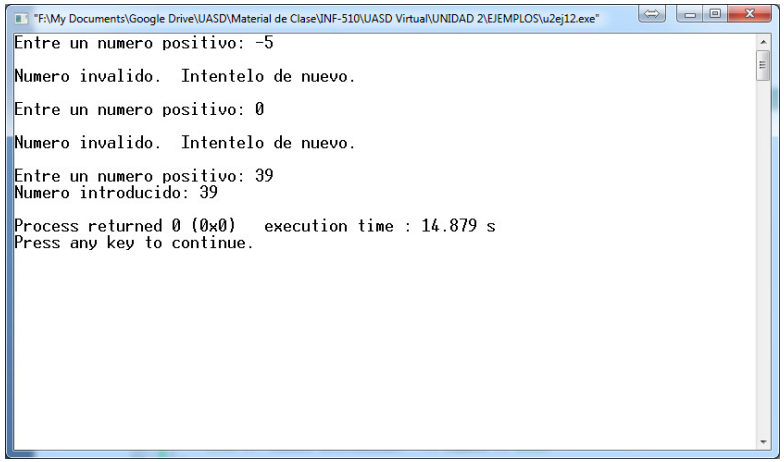
En tal sentido, es importante notar que tanto para el ciclo ***do-while*** como para el ciclo while, no es posible predecir la cantidad de veces que será ejecutado, lo cual no es así para el ciclo ***for***.

En el programa ejemplo, el ciclo solamente finalizará, cuando el valor leído por teclado sea distinto de 's' y también distinto de 'S'.

# Sentencia do-while

- Veamos otro ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int numero;
8
9       do
10      {
11          cout << "Entre un numero positivo: ";
12          cin >> numero;
13
14          if (numero <= 0)
15              cout << "\nNumero invalido.  Intentelo de nuevo.\n\n";
16
17      } while (numero <= 0);
18
19      cout << "Numero introducido: " << numero << endl;
20
21      return 0;
22  }
```



# Sentencia for

- for

```
for (inicio; condicion; modifica)
{
    sentencia 1;
    sentencia 2;
    sentencia N;
}
```

- for

```
for (inicio; condicion; modifica
    sentencia;
```

**Concepto**

**inicio**: inicializa la variable de control del bucle, la cual luego es actualizada por *modifca* en cada ejecución del bucle. En esta parte se debe definir una variable ya sea inicializándola con un valor o declarándola e inicializándola a la vez. Si ésta es declarada entonces la variable es local al cuerpo del ciclo.

**condicion**: funciona como el control del ciclo, evaluando el valor actualizado de *modifca*. Es una expresión lógica que al ser evaluada retorna un valor de verdad falso o verdadero. El ciclo ***for*** se mantendrá en ejecución mientras esta expresión retorne verdadero. El flujo ciclo se detendrá cuando la condición sea falsa o cuando se produzca una terminación forzada mediante una sentencia ***break***.

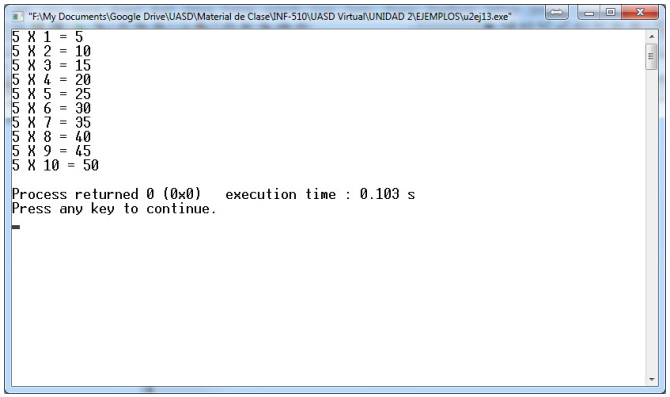
**modifica**: es usada para actualizar el control del ciclo. Es actualizada después de cada ejecución del bloque de código del ***for***. Normalmente en esta parte se define un contador el cual involucra a la variable declarada en *inicio*.

El ciclo ***for*** siempre evalúa al inicio, esto significa que si la condición lógica retorna un valor de verdad falso al ser evaluada, entonces el cuerpo del ciclo no es ejecutado.

# Sentencia for

- Veamos un ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int tabla = 5;
8
9       for (int i = 1; i <= 10; i++)
10      {
11          cout << "5 X " << i << " = "
12               << (5 * i) << endl;
13      }
14
15      return 0;
16  }
```



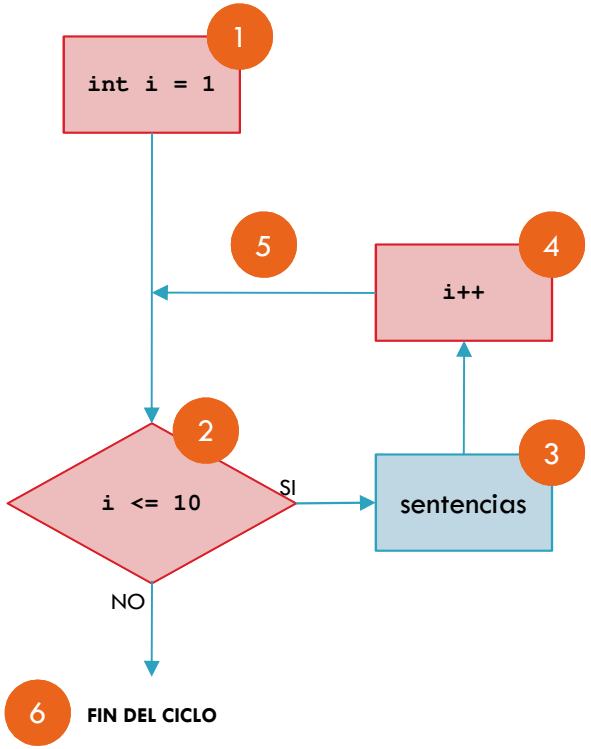

Programa que despliega por pantalla la tabla de multiplicar del 5, desde 1 hasta 10. Notar que declaramos e inicializamos la variable contador en el ***for***, luego del punto y coma (;) se encuentra la condición lógica que determina en que momento o bajo que circunstancias el ciclo ha de terminar. Por último definimos la forma de incremento de la variable, con esto completamos la todo lo que necesitamos para conocer cuantas veces tenemos la intención de ejecutar las sentencias que forman el cuerpo del ciclo.

La sentencia ***for*** a diferencia de las dos sentencias de ciclo anteriores, ***while*** y ***do-while***, es predecible en cuanto a la cantidad de iteraciones que pueden ocurrir.

La forma de ejecución del ***for*** comienza en la primera parte, donde inicializamos la variable, inmediatamente después se procede a evaluar la condición lógica, si el resultado es verdadero entonces el flujo del programa ejecuta todas las sentencias que forman el ciclo, al llegar al final volvemos al inicio del ciclo pero no para ejecutar la primera parte, sino la tercera, donde se actualiza la variable (i++ en nuestro ejemplo), luego se evalúa la condición y si es falsa el ciclo termina, de lo contrario repetimos los pasos.

# Sentencia for

- Explicando su funcionamiento




Las expresiones colocadas en cada paso del flujo están de acuerdo al ejemplo anterior con la finalidad de ilustrar un caso práctico.

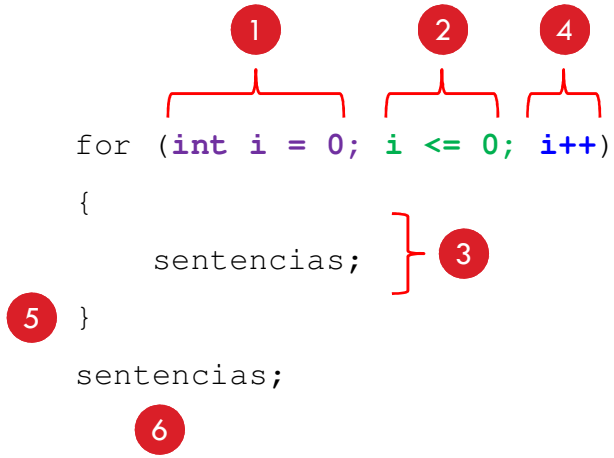
Como pueden observar, el primer componente del ***for*** en ser ejecutado es la parte que inicializa la variable control de la sentencia, en segundo lugar la condición lógica, luego la ejecución del cuerpo del ciclo, retornando al ciclo pero ejecutando la parte de modificación o actualización de la variable control, volvemos con la condición lógica repitiendo estos últimos pasos hasta que retorne un valor de verdad falso.

Veamos los pasos:

1. Inicio del ciclo ***FOR*** (entrada), se ejecuta `int i = 0`
2. Condición lógica, preguntamos si `i <= 0` si es verdadero ir al paso 3, si es falsa ir al paso 6
3. Ejecutar las sentencias del bloque de código del ciclo
4. Actualizar la variable control, en este ejemplo se ejecuta `i++` provocando que la variable aumente su valor en 1
5. Ir al paso 2 (evaluación de nuevo)
6. Fin del ciclo

# Sentencia for

- Desde otro ángulo




1. Inicio del ciclo ***FOR*** (entrada), se ejecuta `int i = 0`
2. Condición lógica, preguntamos si `i <= 0` si es verdadero ir al paso 3, si es falsa ir al paso 6
3. Ejecutar las sentencias del bloque de código del ciclo
4. Actualizar la variable control, en este ejemplo se ejecuta `i++` provocando que la variable aumente su valor en 1
5. Ir al paso 2 (evaluación de nuevo)
6. Fin del ciclo

# Sentencia for

- Desde otro ángulo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       for (int i = 0; i < 500; i++)
8           cout << "No tirare papeles en el curso.\n";
9
10      return 0;
11  }
```



# Sentencia for

- Veamos otro ejemplo (cálculo del factorial)

```cpp
#include <iostream>

using namespace std;

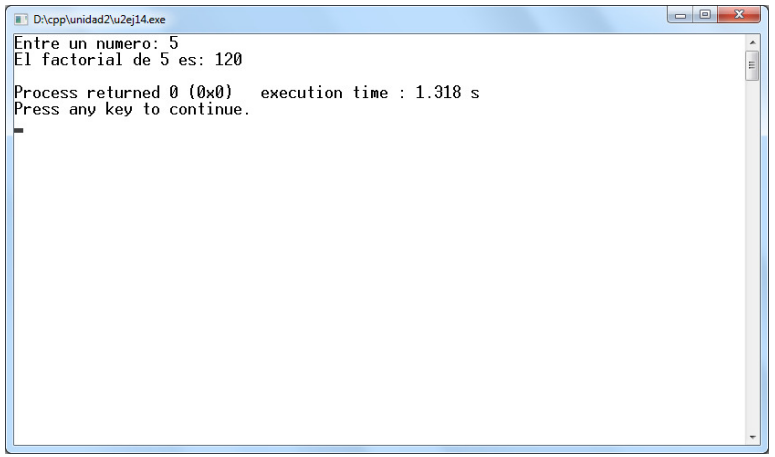
int main(int argc, char** argv)
{
    int numero;
    long factorial = 1;

    cout << "Entre un numero: ";
    cin >> numero;

    for (int cnt = 1; cnt <= numero; cnt++)
    {
        factorial *= cnt;
    }

    cout << "El factorial de " << numero
         << " es: " << factorial << endl;

    return 0;
}
```



# Sentencia for

- Ejemplo anterior mejorado

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int numero;
8       long unsigned long factorial = 1;
9
10      /*****************************************
11       *  Ciclo para garantizar que el valor *
12       *  de la variable numero sea >= cero  *
13       *****************************************/
14      do
15      {
16          cout << "Entre un numero >= cero: ";
17          cin >> numero;
18      } while (numero < 0);
19
20      /*****************************************
21       * Ciclo for para calcular el factorial*
22       *****************************************/
23      for (int cnt = 1; cnt <= numero; cnt++)
24      {
25          factorial *= cnt;
26      }
27
28      cout << "El factorial de " << numero
29           << " es: " << factorial << endl;
30
31      return 0;
32  }
```

Este es el mismo ejemplo anterior pero con algunas mejoras. Note que hemos declarado la variable que almacenará el valor del factorial como una variable sin signo de tipo ***long*** de 16 bytes (long long) para darle mayor capacidad y además colocamos la lectura de la variable ***numero*** dentro de un bucle ***do-while*** el cual se mantendrá ejecutando mientras el valor introducido sea negativo. Le invitamos a probar este programa, modifíquelo, ejecútelo y pruebe con distintos valores.

# Contenido

- Las estructuras de control.
- Sentencia de decisión (if) y operador condicional (?:).
- Sentencia de decisión (switch).
- Sentencias iterativas (while, do-while, for).
- Complementarias (break, continue).

# Sentencias break y continue

break, salir del ciclo

continue, volver al inicio del bucle

# Sentencia break

- La sentencia ***break*** (utilizada anteriormente en la sentencia ***switch***) nos permite también salir del cuerpo del ciclo antes de terminar su ejecución normal
- Puede ser utilizada con cualquiera de las sentencias de repetición:
  - while
  - do-while
  - for
- Salir del ciclo significa que el flujo del programa obviará las demás sentencias que falten por ejecutar y saldrá a la línea inmediatamente después de finalizar el ciclo

# Sentencia break

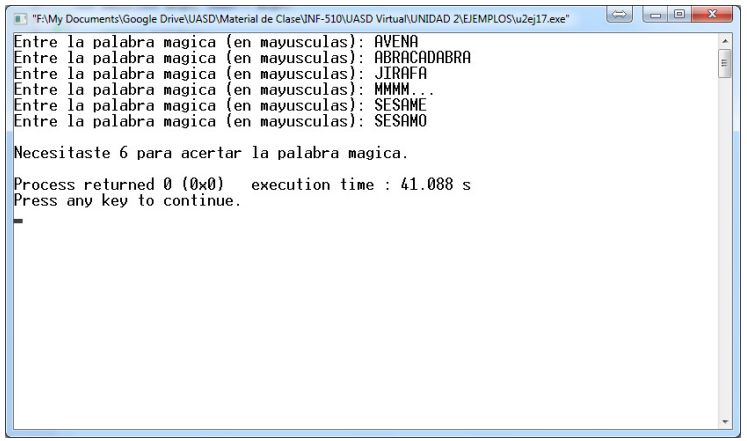
- Veamos un ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       string palabra;
8       const string PALABRA_MAGICA = "SESAMO";
9       int intentos = 0;
10
11      while (true)
12      {
13          cout << "Entre la palabra magica (en mayusculas): ";
14          getline(cin, palabra);
15
16          intentos++;
17
18          if (palabra == PALABRA_MAGICA)
19              break;
20      }
21
22      cout << endl << "Necesitaste " << intentos
23           << " para acertar la palabra magica." << endl;
24
25      return 0;
26  }
```

Este ciclo solo terminará si acertamos a la palabra mágica, pero como tenemos el programa fuente, no será problema pues dicha palabra esta *hardcoded* en el contenido de nuestro ejercicio.

Note algo interesante, estamos usando un ciclo infinito, fíjese en la condición lógica de la sentencia ***while***.

Pruebe introduciendo otros valores, juegue con el código.



# Sentencia continue

- La sentencia ***continue*** la utilizamos dentro de ciclos, esta sentencia permite obviar el resto de las líneas de código que faltan por ejecutar y desvía el flujo hacia el inicio del bucle
- Permite saltar las sentencias del cuerpo del ciclo que están a continuación, yendo directamente a la actualización de la variable de control y a su posterior evaluación por la condición lógica (caso sentencia de repetición ***for***)

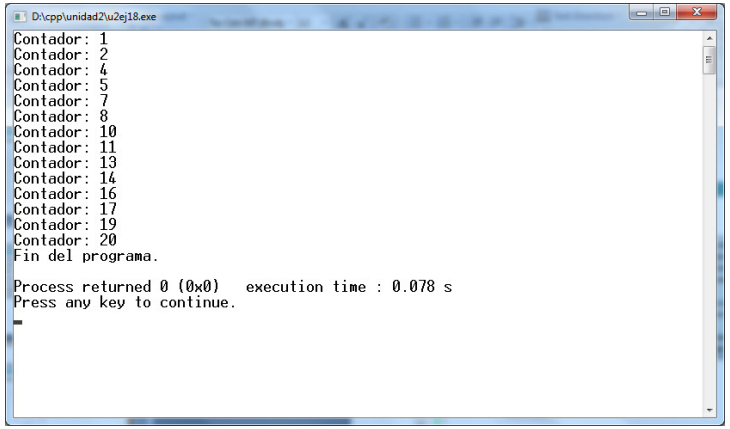
# Sentencia continue

- Veamos un ejemplo

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       int contador = 0;
8       const unsigned int LIMITE = 20;
9
10      /*******************************************
11       * Imprime todos los numeros del 1 al 20    *
12       * excepto aquellos que son multiplos de 3  *
13       *******************************************/
14
15      while (++contador <= LIMITE)
16      {
17          if (contador % 3 == 0)
18              continue;
19
20          cout << "Contador: " << contador << endl;
21      }
22
23      cout << "Fin del programa." << endl;
24
25      return 0;
26  }
```

En este ejemplo podemos ver que si la variable contador al dividirla entre 3, su residuo es cero (divisible por 3) entonces las demás líneas de código son obviadas y el flujo vuelve al inicio del ciclo en donde la variable es incrementada en 1.

Este programa desplegara por pantalla todos los números entre 1 y 20 sin incluir aquellos que sean múltiplos de 3.

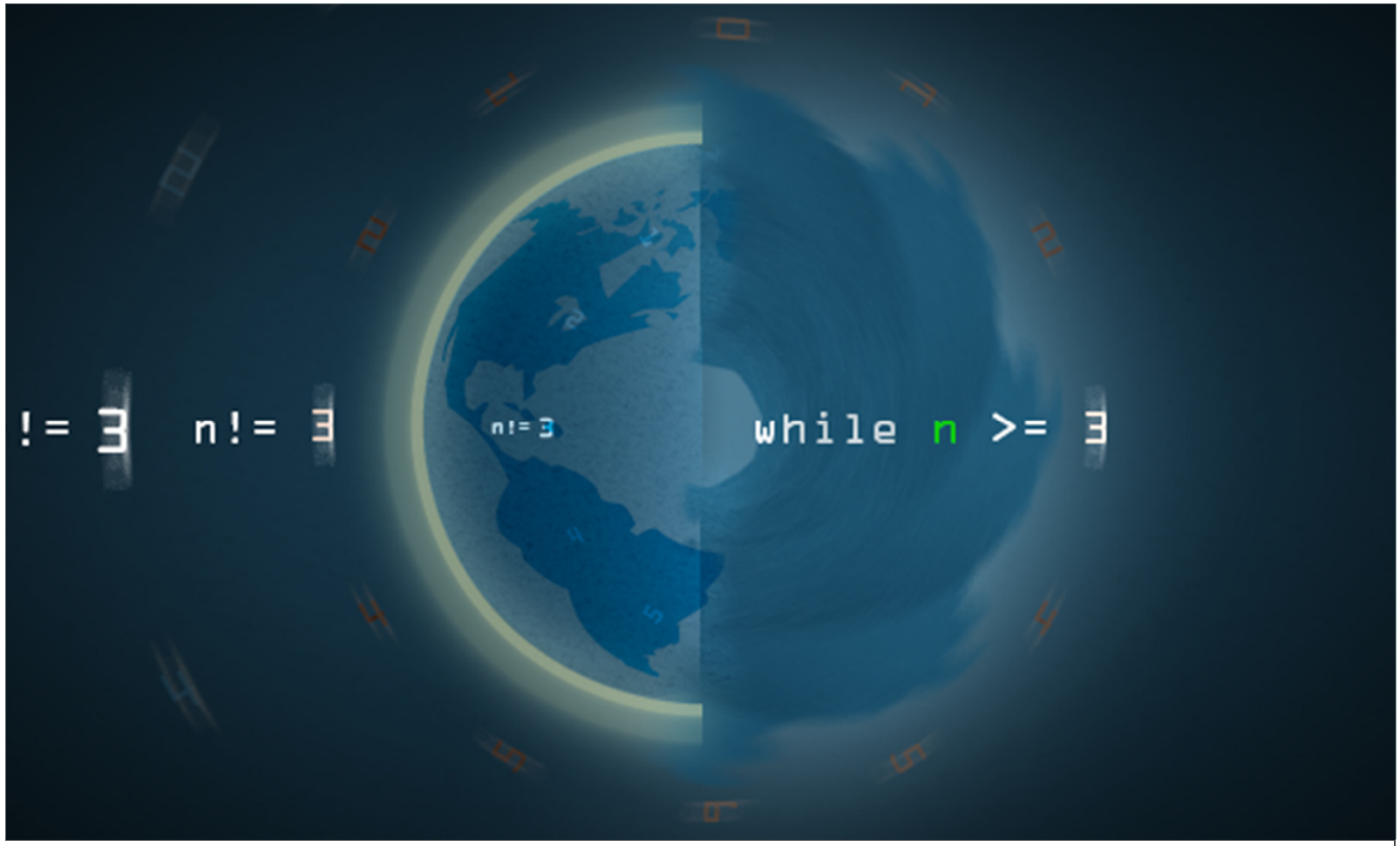


# Resumen

- En esta primera unidad hemos aprendido:
  - Sentencias ***if*** y ***switch*** para toma de decisiones asi como tambien el operador condicional **? :**
  - Repeticion con ciclos ***while***, ***do-while*** y ***for***, siendo este ultimo predecible en cuanto a la cantidad de iteraciones que puede tener mientras que con ***while*** y ***do-while*** no es posible determinar este dato
  - Los ciclos ***while*** evaluan siempre al inicio, provocando con esto que se ejecuten cero o mas veces, mientras que los ciclos ***do-while*** se ejecutan una o mas veces debido a que evaluan al final
  - Tambien aprendimos sobre sentencias que podemos utilizar con los ciclos como la sentencia ***break*** y ***continue***. La sentencia ***break*** permite finalizar un ciclo antes de su terminacion normal mientras que la sentencia ***continue*** obvia o salta el resto del cuerpo del bucle y el flujo es bifurcado al inicio de la sentencia de ciclo

# Bibliografía


- Addison Wesley, Tony Gaddis. (2009). Starting Out with C++ Brief: From Control Structures through Objects. San Francisco, California. 6th Edition.
- Aprende a programar en C++. (2010). Página dedicada a la enseñanza de C++ con libro de libre descarga, recuperado el 3 Junio 2013 de: http://c.conclase.net/curso/index.php
- Página dedicada al lenguaje de programación C++ (2000 – 2014). Tutoriales y recursos de libre descarga, recuperado Enero 2014 en: http://www.cplusplus.com All rights reserved - v3.1
- Paul Deitel, Harvey Deitel (2011). C++ How to Program, New York, 8th Edition, Prentice Hall.




# Unidad 2: Estructuras de Control

Por Félix Ferreiras y Zobiesky Ovalle